

Prompt: Please review our project plan. We need in return a breakdown of how to make this faster, stronger, better. We need this project grounded and optimized so we can execute effectively and build a working minimal viable product.

Project Overview:

Agent Fruity, an agent that can play Fruit Merge Juicy (on IOS and Android, but Agent Fruity would operate through bluestacks 5 or Docker via apk) to a high level of evolved proficiency. The objective is to use evolutionary strategy to create an effective agent that beats human accuracy. We are two people and have 2 windows laptops, and we have one day to create this from scratch.

Master Plan

1. Imports
2. Instance of individual class/object (whatever you wanna call it)
 - I. Elite weights = random
3. For 15 generations:
 - I. Create 4 instances of an individual that is mutated off of the global individual
 - II. For (lifetime of individual = x amount of time) per each individual
 - a. Screenshot current state



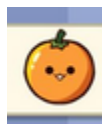
- 1.
- b. If(GAME OVER) Exit this loop early (aka kill the individual that lost)
- c. Segment

1. Current Fruit



a.

2. Next Fruit



a.

3. Score



a.

4. Game State



a.

d. Reduce quality of screenshots

e. Easy OCR for score extraction

1. Score = 425

f. Color ID Based on RGB Match



1.

2. Current Fruit = 4

3. Next Fruit = 5

4. Game state [100][100] (2d array of game state, use color)

ID for each example [6][20] = 5 for color 5 was detected here and 0 for none of the 11 predefined colors.

g. Format into JSON

1. Example:

a. Score: 425

b. Next_Fruit: 5

c. Current_Fruit: 4

d. Board:

i. Fruit: 4

1. X: 3

2. Y: 4

h. Save, Export, and take in JSON as input for Individual

i. The individual outputs x coordinate/"brightness" #1 neuron output

j. Place fruit at the x cord location. Use Action automated method

k. Wait (2 seconds) THIS IS A FIXED TIME WE PICK AS A GLOABAL VARIABLE TO ENSURE THE FRUIT HAS ENOGUH TIME TO FALL AND SETTLE BEFORE THE NEXT SCREENSHOT.

III. Code (regarding all the individuals)

a. From existing JSONs of all individuals, pull final score

b. Compare final scores

c. Elite final score moves on/feedback elite individual into loop

Notes of Concern:

- We need to monitor training through visual outputs and watch the game be played in real time
- We have 2 laptops, how do we do this?
- How do we build an action loop with fruit merge running. How does the ai place the fruits?
- How to classify fruit color ids? We need to extract the game state easily but some fruits share colors (like the pineapple has green leaves similar to the shade of the apple or the watermelon, or the small red berry is somewhat similar to the dragonfruit red)
- How to best get fruit merge to run, blue stacks or docker with apk?
- We have less than 34 hours left and all we have is a plan how do we start and make this real?
- Tech stack? Local or colab? Pytourtch, bluestacks 5? Docker? We need to build a working scrapy version in about 30 hours, we are 2 cs undergrads with 2 laptops that's it.
- For 3.II, regarding "x amount of time," should we use exponential time? Start at 30 seconds and increment by an exponential amount of time as a new generation cycles (Watch max score when it tapers maybe add more seconds then it can improve more)? Like for the first generation, the lifetime of one individual is 30 seconds. For the second generation, the lifetime of one individual is maybe exponentially larger, like 1 minute. Etc. Or should we use Fibonacci time? Or is there a better way to go about the timing?
- How to validate that easy ocr isn't grabbing garbage score numbers and screwing with our evolution?
- Regarding 3.II.b, how should we approach the event where each of the 5 individuals terminate. Should we just respawn and reset that generation? Or is there a better way to handle it?
- How should we split up the code? Is pytorch the thing that splits up code into markdown and whatnot? Is jupyter relevant here?